Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

Relatório de Arquitetura e Implementação

Pablo Felipe Sesco Lopes RA: 143304
Denilson Pires Manzotti RA: 139522

Maringá - Pr
2026

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

Relatório de Arquitetura e Implementação

Maringá - Pr
2026

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

Sumário

1. Visão Geral, Escopo e Metodologia..... 4
1.1 Visão Geral..... 4
1.2 Escopo do Projeto..... 4
1.2.1 Fora do Escopo (Out-of-Scope)..... 5
1.3 Metodologia..... 5
1.3.1 Padrão Arquitetural..... 5
1.3.2 Tecnologias, Linguagens e Recursos Utilizados..... 6
1.3.3 Declaração do Uso Responsável de Inteligência Artificial Generativa..... 7
2. Estrutura e Arquitetura do Projeto..... 8
2.1 Árvore de Diretórios Simplificada..... 8
2.2 Arquitetura de Telas e Navegação..... 9
3. Detalhes de Implementação..... 11
3.1 Estruturação em HTML5..... 11
3.2 Estilização e Layout (Bootstrap 5 + CSS Customizado)..... 11
3.3 Lógica em JavaScript Vanilla..... 11
3.3.1 Os 3 Conceitos Fundamentais do localStorage..... 12
3.3.2 Utilizando localStorage na criação de uma lógica básica de CRUD..... 12
3.3.3 Criação dos CRUDs e utilização de localStorage no projeto..... 15
Referências Bibliográficas..... 17

Maringá - Pr
2026

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

1. Visão Geral, Escopo e Metodologia

1.1 Visão Geral

O Software para Gerenciamento de Petshop é uma aplicação web desenvolvida para otimizar e organizar a rotina operacional de um estabelecimento petshop. A solução permite o controle centralizado das quatro entidades fundamentais do negócio: Clientes, Funcionários, Animais e Atendimentos.

A aplicação oferece uma interface intuitiva e responsiva baseada em HTML, CSS (Bootstrap) e JavaScript Vanilla, permitindo aos usuários realizar operações de inclusão, consulta, alteração e exclusão de dados (CRUD), além de emitir relatórios consolidados para acompanhamento gerencial.

1.2 Escopo do Projeto

- Módulo de Clientes:
    - Cadastro de Cliente armazenando Nome, Telefone, E-mail, RG e CPF.
    - Consulta, Alteração e Remoção de dados de clientes.
    - Geração de Relatório completo de clientes cadastrados.
- Módulo de Funcionários:
    - Cadastro de Funcionário armazenando Nome, Matrícula, Qualificação, Descrição da Função e Carga Horária Semanal.
    - Consulta, Alteração e Remoção de dados de funcionários.
    - Geração de Relatório completo de funcionários cadastrados.
- Módulo de Animais:
    - Cadastro de Animal armazenando Nome, Peso e Altura.
    - Consulta, Alteração e Remoção de dados de animais.
    - Geração de Relatório completo de animais cadastrados.
- Módulo de Atendimentos:

- Registro de Atendimento armazenando Código, Data, Cliente (vínculo), Animal (vínculo) e Funcionário (vínculo).
- Consulta, Alteração e Remoção de atendimentos.
- Geração de Relatório completo de atendimentos realizados.

- Interface e Navegação:
  - Menu principal navegável com acesso a todas as operações e relatórios exigidos.
  - Layout responsivo utilizando o sistema de grid e componentes do Bootstrap.
  - Manipulação de formulários, validações e persistência em memória/localStorage via JavaScript Vanilla.

1.2.1 Fora do Escopo (Out-of-Scope)

- Integração com bancos de dados relacionais ou não-relacionais no lado do servidor (persistência restrita ao cliente/navegador nesta etapa).
- Linguagens e frameworks de backend (como Node.js, PHP, Java ou Python).
- Frameworks JavaScript de frontend (como React, Vue ou Angular).
- Autenticação com controle de acesso, logins ou níveis de permissão.
- Processamento de pagamentos reais ou emissão de notas fiscais.

1.3 Metodologia

1.3.1 Padrão Arquitetural

A aplicação adota um padrão de arquitetura voltado ao desenvolvimento em camada de apresentação (Front-end Single-Page por Módulo).

- Gerenciamento de Estado e Visão: As páginas foram estruturadas de modo a unir as operações operacionais (CRUD) e as exibições analíticas (Relatórios) de cada entidade em telas centrais. O fluxo de dados opera de forma dinâmica, onde os

formulários de cadastro e alteração são renderizados sobre a própria tela através de componentes em janela modal, evitando recarregamentos desnecessários da aplicação.

- Persistência de Dados: A persistência e o manuseio das informações ocorrem no lado do cliente (client-side), estruturando os dados em objetos nativos e manipulando o repositório em memória/navegador (localStorage) para simular o comportamento de um banco de dados relacional nesta fase do projeto.

1.3.2 Tecnologias, Linguagens e Recursos Utilizados

- HTML5: Utilizado para a marcação e estruturação semântica de todas as interfaces (<header>, <nav>, <main>, <section>, <footer>), além de garantir a correta definição dos tipos de inputs e validações primárias dos formulários.

- CSS3 e Bootstrap 5: Responsáveis pela estilização visual e responsividade da interface. Utilizou-se o sistema de malha de 12 colunas (Grid System) do Bootstrap para assegurar a adaptabilidade do layout a diferentes resoluções de telas (desktops, tablets e dispositivos móveis), além do uso de seus componentes padrão como Navbar, Modals, Cards e Tables.

- JavaScript Vanilla (ES6+): Utilizado para prover toda a lógica funcional da aplicação sem a mediação de frameworks de mercado. O código JS é responsável por:
    a. Manipulação dinâmica do Document Object Model (DOM).
    b. Controle e armazenamento das coleções de dados das entidades.
    c. Tratamento de eventos de formulários e botões de ação.
    d. Geração interativa dos relatórios e tabelas de consulta.

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

1.3.3 Declaração do Uso Responsável de Inteligência Artificial Generativa

Em estrita conformidade com a Instrução Normativa GRE Nº 1/2026 da Universidade Estadual de Maringá (UEM), que dispõe sobre o uso ético e responsável de ferramentas de Inteligência Artificial Generativa no âmbito das atividades acadêmicas, declara-se o uso de sistemas de IA como recurso auxiliar e assistido de trabalho, preservando-se integralmente a autoria, a responsabilidade e a contribuição intelectual humana na condução do projeto.
O uso de ferramentas de IA (como modelos de linguagem e geradores de imagem) seguiu rigorosamente os princípios de transparência, supervisão humana e validação prévia de conteúdo estabelecidos pela instituição:

1. Apoio à Organização Textual e Documentação: Utilizaram-se ferramentas de processamento de linguagem natural (PLN) exclusivamente como apoio técnico assistido para a estruturação do documento, formatação em Markdown e revisão gramatical/estilística. Todo o conteúdo técnico, regras de negócio e conceitos acadêmicos foram integralmente revisados, validados e supervisionados pelos integrantes da equipe, garantindo a precisão e a veracidade das informações apresentadas.

2. Construção de Diagramas e Elementos Visuais: Modelos generativos baseados em prompts de texto foram empregados na concepção visual e renderização dos diagramas de arquitetura e navegação do sistema (mapa do site). As representações geradas foram submetidas à análise crítica da equipe para assegurar que não contivessem inconsistências ou divergências com a estrutura real desenvolvida no código.

Maringá - Pr
2026

3. Garantia de Autoria e Integridade Acadêmica: Em atendimento às vedações expressas no Art. 5º da referida normativa, a IA não substituiu a elaboração autoral nem a tomada de decisões técnicas do projeto. A autoria e a implementação do código-fonte (HTML, CSS e JavaScript Vanilla) são de responsabilidade direta dos discentes autores. Além disso, nenhuma informação sigilosa ou dado pessoal sensível foi submetido às ferramentas de IA durante o processo.

2. Estrutura e Arquitetura do Projeto

Para garantir manutenibilidade e evitar duplicação de código HTML, adotou-se uma arquitetura baseada em módulos unificados por entidade. Em vez de criar páginas separadas para cada operação individual (Inclusão, Alteração, Exclusão e Consulta), cada entidade possui uma página dedicada que centraliza suas operações ativas e relatórios através do uso de modais e componentes dinâmicos do Bootstrap.

2.1 Árvore de Diretórios Simplificada

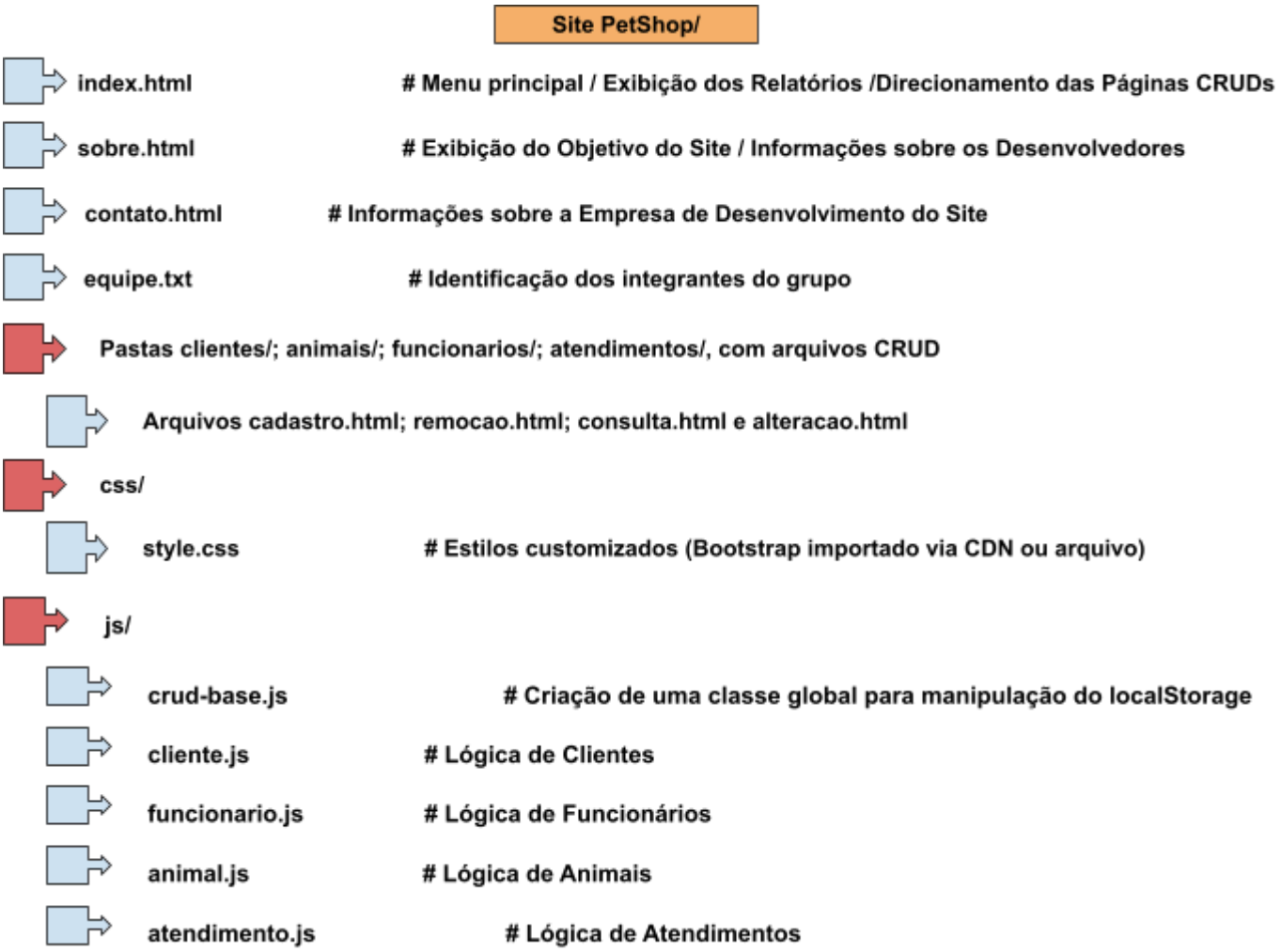


Figura 1 – Representação visual da árvore de diretórios do projeto.

2.2 Arquitetura de Telas e Navegação

A navegação do sistema foi desenhada para ser simples, intuitiva e acessível a partir de qualquer ponto da aplicação através do menu principal (Navbar) persistente.

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro



Figura 2 – Diagrama de arquitetura web modular para sistema de pet shop.

Fonte: Gerada artificialmente por Google Gemini (versão Flash 1.5/2.0), a partir do prompt: *"A clean minimalist web architecture diagram for a petshop management site. Top box labeled "Página Inicial (Home)", branching down into four colored boxes labeled "Clientes", "Funcionários", "Animals", and "Atendimentos". Below each module box are four sub-items in light grey: "Novo", "Editar", "Excluir", and "Relatório Tabela". Modern UI vector diagram style on a light background. Clear design, no extra text or directory paths."*, em 31 out. 2026.

3. Detalhes de Implementação

3.1 Estruturação em HTML5

- Tags html utilizadas:
    - Estrutura básica: html, head, body, title, meta, link, style, script
    - Estrutura e organização: div, span, header, nav, fieldset, form
    - Títulos e textos: h1, h3, h5, p, br, pre, code, caption
    - Formatação de texto: strong, em, i, mark
    - Formulários e entradas: input, button, select, option, datalist, textarea
    - Listas: ol, ul, li
    - Tabelas: table, thead, tbody, tfoot, tr, th, td
    - Imagens e recursos externos: img
- Formulários estruturados com type adequado (email, tel, number, date) e validação via atributo required.

3.2 Estilização e Layout (Bootstrap 5 + CSS Customizado)

- Sistema de Grid: Aplicação de classes .row e .col-md-* para garantir responsividade em desktops e celulares.
- Componentes: Uso do Navbar para menu persistente, Tables estilizadas para relatórios e Modais para confirmações e edições rápidas.

3.3 Lógica em JavaScript Vanilla

No desenvolvimento web, o HTML e o JavaScript da página rodam diretamente no navegador. Como as variáveis do JavaScript somem da memória assim que a página é

atualizada ou fechada, usamos o localStorage: um banco de dados simples (chave-valor) mantido pelo próprio navegador.

3.3.1 Os 3 Conceitos Fundamentais do localStorage

Tudo no localStorage gira em torno de duas regras essenciais:

1. Ele só salva texto (String): Você não consegue salvar objetos ou arrays do JavaScript diretamente.
2. JSON.stringify(): Converte arrays e objetos JavaScript em uma string de texto para poder salvar.
3. JSON.parse(): Pega o texto do localStorage e o transforma de volta em objetos/arrays JavaScript para você ler e manipular.

A interface do localStorage fornece métodos simples para manipular os dados:

- localStorage.setItem('chave', 'valor') — Salva ou atualiza um dado.
- localStorage.getItem('chave') — Lê o valor associado àquela chave.
- localStorage.removeItem('chave') — Remove uma chave específica.
- localStorage.clear() — Apaga todos os dados salvos por aquela origem/domínio.

3.3.2 Utilizando localStorage na criação de uma lógica básica de CRUD

- Create (Criar/Adicionar)

Quando o usuário envia um formulário:

1. Capturamos os valores dos campos do formulário.
2. Criamos um novo objeto contendo um identificador único (geralmente Date.now() ou um contador) e os dados inseridos.
3. Adicionamos (push) o objeto ao array em memória.
4. Salvamos o array atualizado no localStorage.
5. Renderizamos novamente a lista na tela.

```javascript
function criarItem(nome, email) {
  const novoNovoItem = {
    id: Date.now(), // Gera um ID único baseado no timestamp atual
    nome: nome,
    email: email
  };

  listaDados.push(novoNovoItem); // 1. Adiciona na memória
  salvarNoLocalStorage();        // 2. Persiste no localStorage
  renderizarLista();             // 3. Atualiza a tela
}
```

- Read (Ler e Exibir na Tela)

Para exibir os dados no HTML:

1. Limpamos o elemento contêiner (ex: <tbody> ou <ul>).
2. Percorremos o array listaDados (usando forEach ou map).
3. Para cada item, geramos o HTML correspondente com os dados e botões de Ação (Editar/Excluir) passando o id do item.

```javascript
function renderizarLista() {
  const conteiner = document.getElementById('listaContainer');
  conteiner.innerHTML = ''; // Limpa a lista atual

  listaDados.forEach(item => {
    conteiner.innerHTML += `
      <div id="item-${item.id}">
```

```
        <p>${item.nome} - ${item.email}</p>
        <button onclick="prepararEdicao(${item.id})">Editar</button>
        <button onclick="deletarItem(${item.id})">Excluir</button>
      </div>
    `;
  });
}
```

- Update (Atualizar)

A edição geralmente acontece em dois passos:

1. Localizar: Encontramos o item no array usando listaDados.find(item => item.id === id) e preenchemos os campos do formulário com os valores atuais.
2. Salvar Alteração: Substituímos as propriedades do objeto no array e salvamos novamente.

JavaScript

```
function atualizarItem(id, novoNome, novoEmail) {
  // Procura o índice do item no array
  const index = listaDados.findIndex(item => item.id === id);

  if (index !== -1) {
    listaDados[index].nome = novoNome;
    listaDados[index].email = novoEmail;

    salvarNoLocalStorage(); // Persiste a alteração
    renderizarLista();      // Atualiza a tela
  }
}
```

- Delete (Remover)

Para apagar um registro:

1. Usamos filter() para criar um novo array contendo todos os itens, exceto aquele com o id selecionado.
2. Atualizamos listaDados com esse novo array.
3. Salvamos no localStorage e renderizamos a tela.

```javascript
function deletarItem(id) {
  // Mantém apenas os itens que têm ID diferente do selecionado
  listaDados = listaDados.filter(item => item.id !== id);

  salvarNoLocalStorage(); // Persiste no localStorage
  renderizarLista();      // Atualiza a tela
}
```

3.3.3 Criação dos CRUDs e utilização de localStorage no projeto

A implementação dos CRUDs no projeto adota a manipulação do localStorage para a persistência de dados no lado do cliente, simulando o fluxo de um banco de dados relacional. Como a API nativa do localStorage armazena exclusivamente texto, os fluxos de criação, leitura, atualização e exclusão exigem a conversão estrutural dos dados em memória; arrays e objetos são transformados em strings através do método JSON.stringify() no momento da gravação, e reconstruídos como objetos interativos usando JSON.parse() na etapa de leitura. Seguindo metodologias de construção passo a passo demonstradas em tutoriais fundamentais de manipulação de DOM e repositórios locais em JavaScript, as operações atuam primeiramente em um array temporário (utilizando métodos como push, findIndex e filter) antes da efetivação via setItem.

Na estruturação das interfaces, o sistema adota uma arquitetura modularizada em páginas HTML distintas para cada funcionalidade da entidade. Usando o fluxo de Clientes como referência, o ecossistema divide as responsabilidades físicas em páginas específicas: cadastro.html atua na inserção de novos registros, consulta.html exibe as listagens dinâmicas filtráveis, alteracao.html carrega dados pré-existentes para edição e remocao.html cuida da exclusão definitiva, incluindo proteções como modais de confirmação para ações irreversíveis. Esta abordagem substitui as tradicionais aplicações de página única (SPA) baseadas inteiramente em formulários pop-up, preferindo rotas e redirecionamentos enriquecidos com parâmetros de URL (ex: ?cpf=) para o transporte de chaves de identificação primária.

A coesão do código é garantida pela separação de responsabilidades na camada lógica. O arquivo cliente.js isola as operações de infraestrutura na classe genérica StorageService, encarregada de extrair, varrer, adicionar e remover itens pontuais do registro salvo. Em paralelo, controladoras de visualização especializadas (ClienteController, ClienteConsultaController e ClienteRemocaoController) limitam-se a extrair dados via FormData, renderizar dinamicamente o HTML na tabela e capturar cliques. Quando um cliente é salvo ou alterado, as funções preventivamente limpam os inputs e redirecionam automaticamente o usuário de volta à interface de consulta.

3.3.4 Geração de Relatórios Completos usando localStorage

- Leitura e Parse SEGURO do localStorage

Como o localStorage armazena apenas dados em formato textual (String), foi criada uma função auxiliar (obterDados) que realiza a leitura e o parse JSON seguro. Caso a chave não exista no navegador ou contenha dados corrompidos, a função trata a exceção e retorna um array vazio ([]), prevenindo quebras de execução (runtime errors).

- Mapeamento Completo de Campos e Normalização de Nomes

Para garantir que todos os campos preenchidos nos formulários de cadastro fossem representados no relatório, foram mapeadas as colunas equivalentes de cada formulário:

- Clientes: ID, Nome, CPF, RG, Telefone, E-mail, Data de Nascimento, Gênero, Cidade, Preferências de Propaganda (E-mail/WhatsApp), Quantidade de Pets, Logradouro e Anotações.
- Atendimentos: ID/Número, Animal, Tutor, Funcionário, Data do Atendimento e Anotações.
- Animais: ID, Nome, Tutor, CPF do Tutor, SinPatinhas (RG Animal), Peso, Altura, Espécie, Gênero, Data de Nascimento, Microchip e Anotações.
- Funcionários: ID, Nome, CPF, RG, Matrícula, Telefone, E-mail, Data de Nascimento, Gênero, Cargo, Qualificação, Carga Horária e Descrição do Cargo.

Como os atributos salvos nos objetos podem variar dependendo do estado do controlador (ex: usando o nome do campo dataNascimento ou a propriedade do input data), a lógica utiliza o operador de coalescência/fallback (c.dataNascimento || c.data) para capturar o dado corretamente.

- Tratamento de Valores Ausentes ("não informado")

Para atender ao requisito de identificação de campos não preenchidos, foi desenvolvida a função de formatação formatar(valor). Ela avalia se a informação é nula, indefinida (undefined) ou uma string vazia (ou contendo apenas espaços). Caso seja verdadeira a ausência do dado, retorna dinamicamente o trecho HTML:
HTML

```HTML
<em class="text-muted">não informado</em>
```

Isso garante diferenciação visual e clareza para quem está consultando a tabela.

- Renderização Dinâmica e Indicadores de Quantidade

Assim que o evento DOMContentLoaded é disparado:

1. O script busca os registros de cada entidade no localStorage.
2. Atualiza o badge de contagem no cabeçalho de cada card de entidade (ex: X registro(s)).
3. Se a entidade não possuir registros, exibe uma mensagem informativa centralizada indicando a ausência de dados.
4. Se houver registros, percorre a lista gerando as linhas da tabela (<tr> e <td>) com os dados formatados em tempo de execução.

Universidade Estadual De Maringá - UEM
Centro De Tecnologia - CTC
Departamento De Informática - DIN
Curso: Engenharia De Software - ESW
Disciplina: 9907/31 - Programação de Sistemas Web
Professor: Andre Felipe Ribeiro Cordeiro

Referências Bibliográficas

- **GOOGLE. Gemini (versão Flash).** Ferramenta computacional baseada em inteligência artificial. Google, 2026. Disponível em: google.com. Acesso em: 31 out. 2026.
- **LEONID, Fernando. CRUD Completo do Zero com JavaScript e LocalStorage | Projeto 08. 30 jun. 2021.** 1 vídeo (1 h 45 min 57 s). Publicado pelo canal Fernando Leonid. Disponível em: https://www.youtube.com/watch?v=_HEIqE_qqbQ. Acesso em: 25 out. 2026.
- **HTML Tutorial.** W3Schools, [s. l.], 2026. Disponível em: https://www.w3schools.com/html/default.asp. Acesso em: 17 out. 2026.
- **CSS Tutorial.** W3Schools, [s. l.], 2026. Disponível em: https://www.w3schools.com/css/default.asp. Acesso em: 17 out. 2026.
- **BOOTSTRAP 4 Tutorial.** W3Schools, [s. l.], 2026. Disponível em: https://www.w3schools.com/bootstrap4/default.asp. Acesso em: 17 out. 2026.
- **JAVASCRIPT Tutorial.** W3Schools, [s. l.], 2026. Disponível em: https://www.w3schools.com/js/default.asp. Acesso em: 24 out. 2026.
- **WEB Storage API.** W3Schools, [s. l.], 2026. Disponível em: https://www.w3schools.com/js/js_api_web_storage.asp. Acesso em: 24 out. 2026.